

Quiz2_solutions

```
if (!requireNamespace("caret", quietly = TRUE)) install.packages("caret")
if (!requireNamespace("neuralnet", quietly = TRUE)) install.packages("neuralnet")

library(caret)
library(neuralnet)
```

#Task 1. Regression Task with airquality

Q1. Data cleaning, split, and standardization

```
data(airquality)
aq <- na.omit(airquality)
cat("Number of observations remaining after removing missing values:", nrow(aq), "\n")
```

Number of observations remaining after removing missing values: 111

```
set.seed(123)
idx_reg <- sample(seq_len(nrow(aq)), size = floor(0.8 * nrow(aq)))

train_raw <- aq[idx_reg, ]
test_raw <- aq[-idx_reg, ]

cat("Training size:", nrow(train_raw), "\n")
```

Training size: 88

```
cat("Testing size:", nrow(test_raw), "\n")
```

Testing size: 23

```
train_center <- sapply(train_raw, mean)
train_scale <- sapply(train_raw, sd)

train_reg <- as.data.frame(scale(train_raw, center = train_center, scale = train_scale))
test_reg <- as.data.frame(scale(test_raw, center = train_center, scale = train_scale))

y_mean <- train_center["Ozone"]
y_sd <- train_scale["Ozone"]

x_reg_names <- setdiff(names(train_reg), "Ozone")
```

Q2. NN with no hidden layer, SSE, linear output

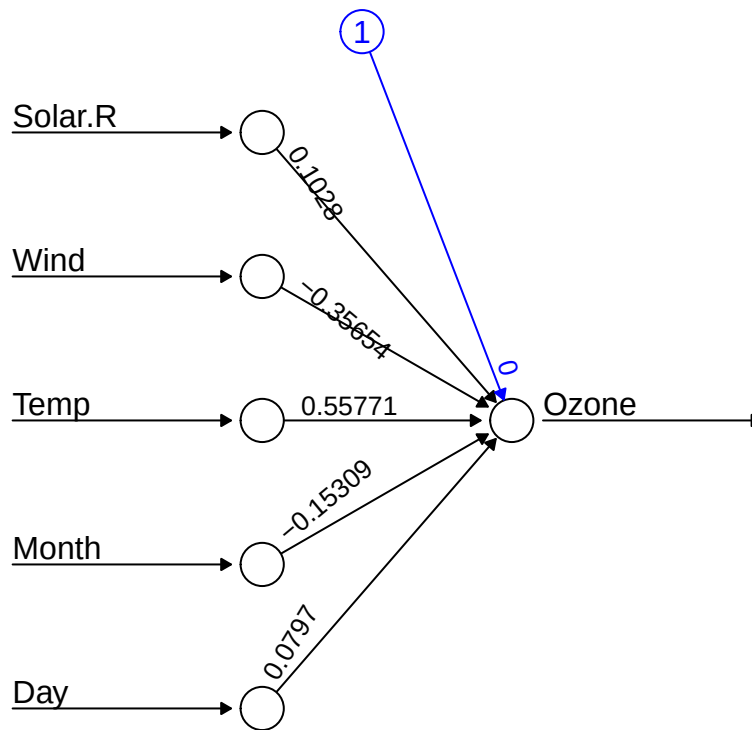
```

set.seed(123)

nn_reg_0 <- neuralnet(
  Ozone ~ .,
  data = train_reg,
  hidden = 0,
  err.fct = "sse",
  linear.output = TRUE,
  stepmax = 1e6
)

plot(nn_reg_0, rep = "best")

```



Error: 15.901499 Steps: 58

```

pred_reg_0_scaled <- as.vector(compute(nn_reg_0, test_reg[, x_reg_names])$net.result)

# Back-transform to original Ozone scale
pred_reg_0 <- pred_reg_0_scaled * y_sd + y_mean

mse_reg_0 <- mean((test_raw$Ozone - pred_reg_0)^2)
cat("Test MSE for Model 1 (no hidden layer):", mse_reg_0, "\n")

```

```
## Test MSE for Model 1 (no hidden layer): 348.2305
```

Q3. NN with one hidden layer (3 neurons), logistic activation, SSE, linear output

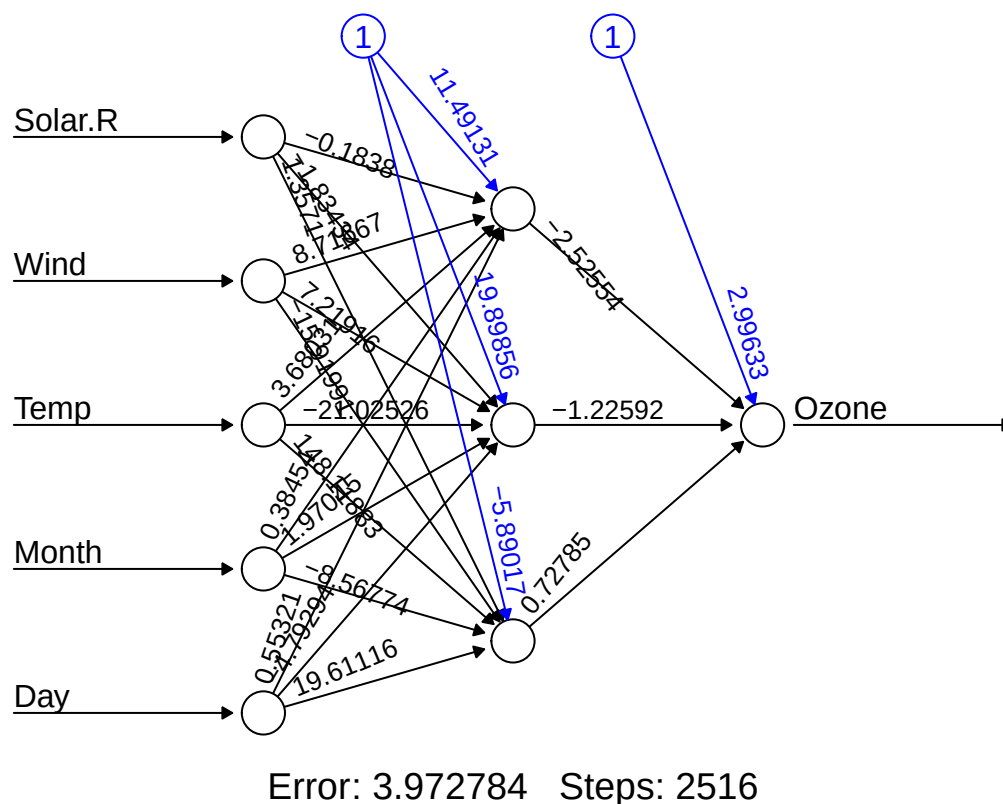
```

set.seed(123)

nn_reg_1 <- neuralnet(
  Ozone ~ .,
  data = train_reg,
  hidden = 3,
  act.fct = "logistic",
  err.fct = "sse",
  linear.output = TRUE,
  stepmax = 1e6
)

plot(nn_reg_1, rep = "best")

```



```

pred_reg_1_scaled <- as.vector(compute(nn_reg_1, test_reg[, x_reg_names])$net.result)

# Back-transform to original Ozone scale
pred_reg_1 <- pred_reg_1_scaled * y_sd + y_mean

mse_reg_1 <- mean((test_raw$Ozone - pred_reg_1)^2)
cat("Test MSE for Model 2 (one hidden layer with 3 neurons):", mse_reg_1, "\n")

```

```
## Test MSE for Model 2 (one hidden layer with 3 neurons): 480.7752
```

Q4. Comparison of the two regression models

```
reg_compare <- data.frame(
  Model = c("No hidden layer", "One hidden layer (3 neurons)"),
  Test_MSE = c(mse_reg_0, mse_reg_1)
)

reg_compare
```

```
##                Model Test_MSE
## 1                No hidden layer 348.2305
## 2 One hidden layer (3 neurons) 480.7752
```

```
if (mse_reg_0 < mse_reg_1) {
  cat("The no-hidden-layer model performs better on this split because it has the lower Test MSE.\n")
} else if (mse_reg_1 < mse_reg_0) {
  cat("The one-hidden-layer model performs better on this split because it has the lower Test MSE.\n")
} else {
  cat("The two models have the same Test MSE on this split.\n")
}
```

The no-hidden-layer model performs better on this split because it has the lower Test MSE.

The hidden-layer model is more flexible and can capture nonlinear patterns, but with a relatively small dataset it may also overfit the training data, leading to worse performance on the testing set.

#Task 2. Classification Task with Titanic Data

Q5. Data cleaning, dummy coding, and split

```
tit <- read.csv("Titanic2.csv", stringsAsFactors = FALSE)

tit <- tit[, c("Survived", "Pclass", "Sex", "Age")]
tit <- na.omit(tit)

cat("Number of observations remaining after removing missing values:", nrow(tit), "\n")
```

Number of observations remaining after removing missing values: 714

```
tit$Survived <- as.numeric(as.character(tit$Survived))
tit$Pclass <- factor(tit$Pclass)
tit$Sex <- factor(tit$Sex)

X_cls <- model.matrix(~ Pclass + Sex + Age, data = tit)[, -1]
cls_data <- data.frame(Survived = tit$Survived, X_cls)

head(cls_data)
```

```
##   Survived Pclass2 Pclass3 Sexmale Age
## 1         0         0         1         1 22
## 2         1         0         0         0 38
## 3         1         0         1         0 26
## 4         1         0         0         0 35
## 5         0         0         1         1 35
## 7         0         0         0         1 54
```

```

set.seed(123)
idx_cls <- createDataPartition(cls_data$Survived, p = 0.80, list = FALSE)

train_cls <- cls_data[idx_cls, ]
test_cls <- cls_data[-idx_cls, ]

cat("Training size:", nrow(train_cls), "\n")

```

Training size: 572

```

cat("Testing size:", nrow(test_cls), "\n")

```

Testing size: 142

Q6. NN with no hidden layer, SSE, nonlinear output (linear.output = FALSE)

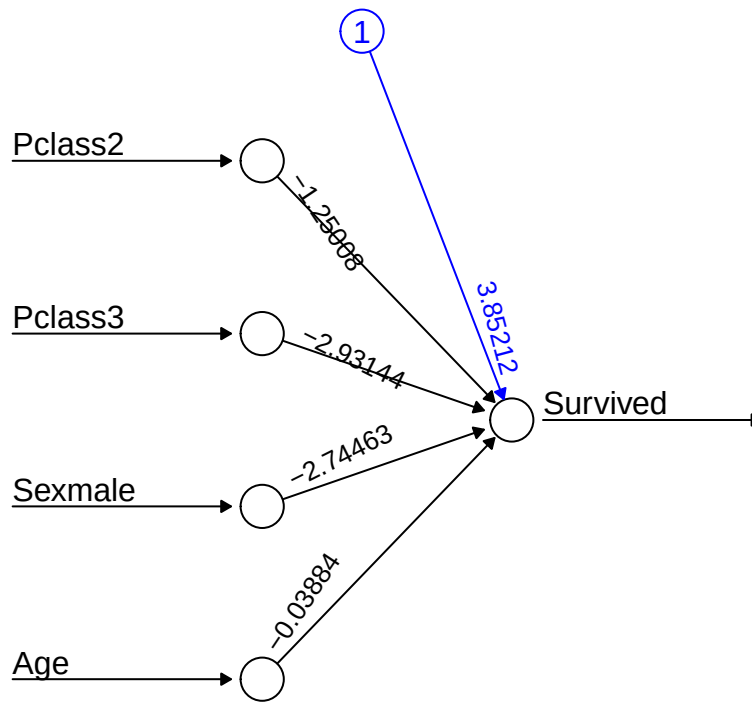
```

set.seed(123)

nn_cls_0 <- neuralnet(
  Survived ~ .,
  data = train_cls,
  hidden = 0,
  err.fct = "sse",
  linear.output = FALSE,
  stepmax = 1e6
)

plot(nn_cls_0, rep = "best")

```



Error: 41.906564 Steps: 343

```

x_cls_names <- setdiff(names(train_cls), "Survived")

prob_cls_0 <- as.vector(compute(nn_cls_0, test_cls[, x_cls_names])$net.result)
pred_cls_0 <- ifelse(prob_cls_0 > 0.5, 1, 0)

cm_cls_0 <- confusionMatrix(
  factor(pred_cls_0, levels = c(0, 1)),
  factor(test_cls$Survived, levels = c(0, 1)),
  positive = "1"
)

cm_cls_0

```

```

## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0    1
##           0 73 20
##           1  6 43
##
##           Accuracy : 0.8169
##           95% CI : (0.7433, 0.8768)
##           No Information Rate : 0.5563
##           P-Value [Acc > NIR] : 5.201e-11
##

```

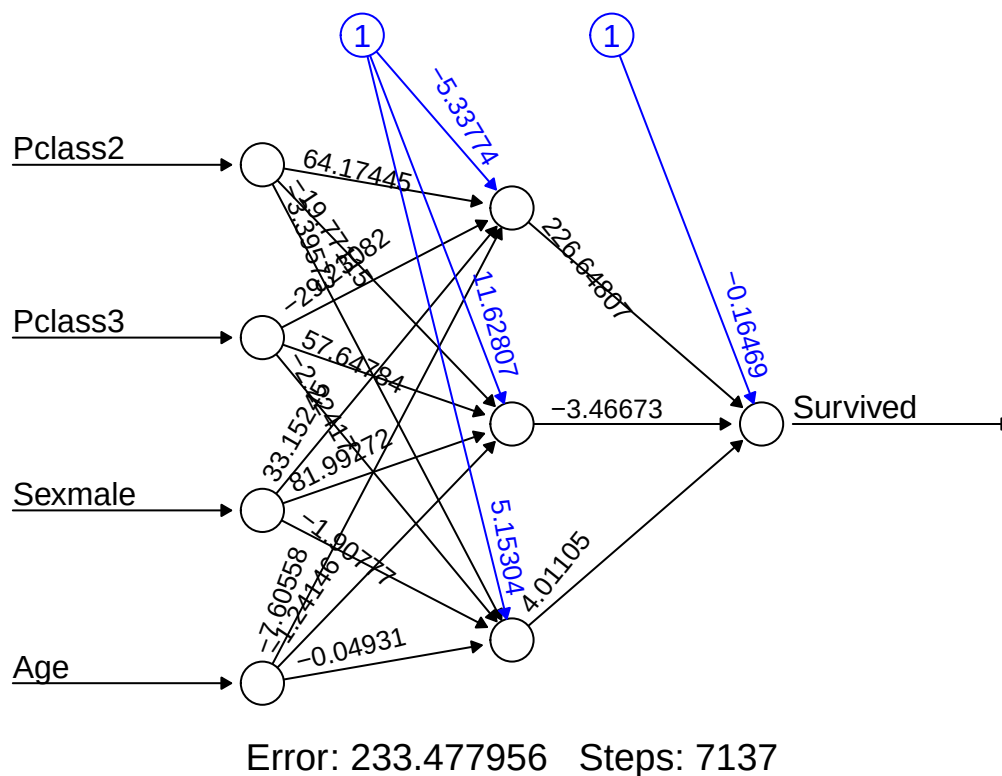
```
##           Kappa : 0.6206
##
## Mcnemar's Test P-Value : 0.01079
##
##           Sensitivity : 0.6825
##           Specificity : 0.9241
##           Pos Pred Value : 0.8776
##           Neg Pred Value : 0.7849
##           Prevalence : 0.4437
##           Detection Rate : 0.3028
##           Detection Prevalence : 0.3451
##           Balanced Accuracy : 0.8033
##
##           'Positive' Class : 1
##
```

Q7. NN with one hidden layer (3 neurons), logistic activation, CE, nonlinear output

```
set.seed(123)

nn_cls_1 <- neuralnet(
  Survived ~ .,
  data = train_cls,
  hidden = 3,
  act.fct = "logistic",
  err.fct = "ce",
  linear.output = FALSE,
  stepmax = 1e6
)

plot(nn_cls_1, rep = "best")
```



```

prob_cls_1 <- as.vector(compute(nn_cls_1, test_cls[, x_cls_names])$net.result)
pred_cls_1 <- ifelse(prob_cls_1 > 0.5, 1, 0)

cm_cls_1 <- confusionMatrix(
  factor(pred_cls_1, levels = c(0, 1)),
  factor(test_cls$Survived, levels = c(0, 1)),
  positive = "1"
)

cm_cls_1

```

```

## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0    1
##           0  77  23
##           1   2  40
##
##               Accuracy : 0.8239
##               95% CI : (0.7512, 0.8827)
##       No Information Rate : 0.5563
##       P-Value [Acc > NIR] : 1.424e-11
##
##               Kappa : 0.6309
##

```

```
## McNemar's Test P-Value : 6.334e-05
##
##      Sensitivity : 0.6349
##      Specificity : 0.9747
##      Pos Pred Value : 0.9524
##      Neg Pred Value : 0.7700
##      Prevalence : 0.4437
##      Detection Rate : 0.2817
##      Detection Prevalence : 0.2958
##      Balanced Accuracy : 0.8048
##
##      'Positive' Class : 1
##
```

Q8. Comparison of the two classification models

```
cls_compare <- data.frame(
  Model = c("No hidden layer (SSE)", "One hidden layer (3 neurons, CE)"),
  Accuracy = c(
    unname(cm_cls_0$overall["Accuracy"]),
    unname(cm_cls_1$overall["Accuracy"])
  ),
  Sensitivity = c(
    unname(cm_cls_0$byClass["Sensitivity"]),
    unname(cm_cls_1$byClass["Sensitivity"])
  ),
  Specificity = c(
    unname(cm_cls_0$byClass["Specificity"]),
    unname(cm_cls_1$byClass["Specificity"])
  )
)

cls_compare
```

```
##              Model  Accuracy Sensitivity Specificity
## 1      No hidden layer (SSE) 0.8169014   0.6825397   0.9240506
## 2 One hidden layer (3 neurons, CE) 0.8239437   0.6349206   0.9746835
```

```
acc0 <- unname(cm_cls_0$overall["Accuracy"])
acc1 <- unname(cm_cls_1$overall["Accuracy"])
sens0 <- unname(cm_cls_0$byClass["Sensitivity"])
sens1 <- unname(cm_cls_1$byClass["Sensitivity"])
spec0 <- unname(cm_cls_0$byClass["Specificity"])
spec1 <- unname(cm_cls_1$byClass["Specificity"])

if (acc0 > acc1) {
  cat("Model 1 has the higher overall accuracy on this split.\n")
} else if (acc1 > acc0) {
  cat("Model 2 has the higher overall accuracy on this split.\n")
} else {
  cat("The two models have the same overall accuracy on this split.\n")
}
```

```
## Model 2 has the higher overall accuracy on this split.
```

```
cat("Model 1 Sensitivity:", sens0, "\n")
```

```
## Model 1 Sensitivity: 0.6825397
```

```
cat("Model 2 Sensitivity:", sens1, "\n")
```

```
## Model 2 Sensitivity: 0.6349206
```

```
cat("Model 1 Specificity:", spec0, "\n")
```

```
## Model 1 Specificity: 0.9240506
```

```
cat("Model 2 Specificity:", spec1, "\n")
```

```
## Model 2 Specificity: 0.9746835
```

Overall, if we mainly focus on accuracy and specificity, then Model 2 performs slightly better on this split. On the other hand, if correctly identifying survivors is more important, then Model 1 may be preferred because of its higher sensitivity.

Therefore, the better model depends on which metric is considered more important. For overall performance on this split, Model 2 is slightly better, but it comes with a trade-off of lower sensitivity.